

IT PROFESSIONAL PORTFOLIO

Cloud Engineering and Security Projects

PROJECT 34

Replace SSH with Session Manager Run As and CLI Access

AWS Systems Manager | Session Manager | EC2 | AWS CLI | CloudFormation

AUTHOR	Michael Salazar
ENVIRONMENT	Personal AWS Lab
VERSION	v1.0.0
RELEASE DATE	July 23, 2026
STATUS	Complete

Executive Summary

Logged into **TestAccount1** with **AdministratorAccess**, opened **AWS Systems Manager Session Manager**, edited preferences to enable **Run As support for Linux instances**, and set **ec2-user** as the default operating-system account for Amazon Linux sessions. Then created the **SLAW** CloudFormation stack from the provided template, validated the SLAW and SLAW2 private EC2 instances, connected to SLAW through Session Manager, confirmed the session ran as **ec2-user**, and used the AWS CLI command **aws ssm start-session --target** from SLAW to start a Session Manager connection to SLAW2. This demonstrated a non-SSH connection path both from the console and from the command line.

SECURITY
OUTCOME

This lab expands Session Manager from browser-based access into controlled operating-system identity and command-line access. It avoids SSH keys, public SSH exposure, and jump-box dependency.

Business Problem

SSH is a common administrative path, but exposed SSH is heavily attacked and SSH key management becomes painful at scale. Session Manager shifts administrative access into AWS identity, managed instance communication, and auditable control-plane access. This lab focuses on two operational improvements: using Run As to control the Linux user account and using the AWS CLI to start a Session Manager connection.

Objectives

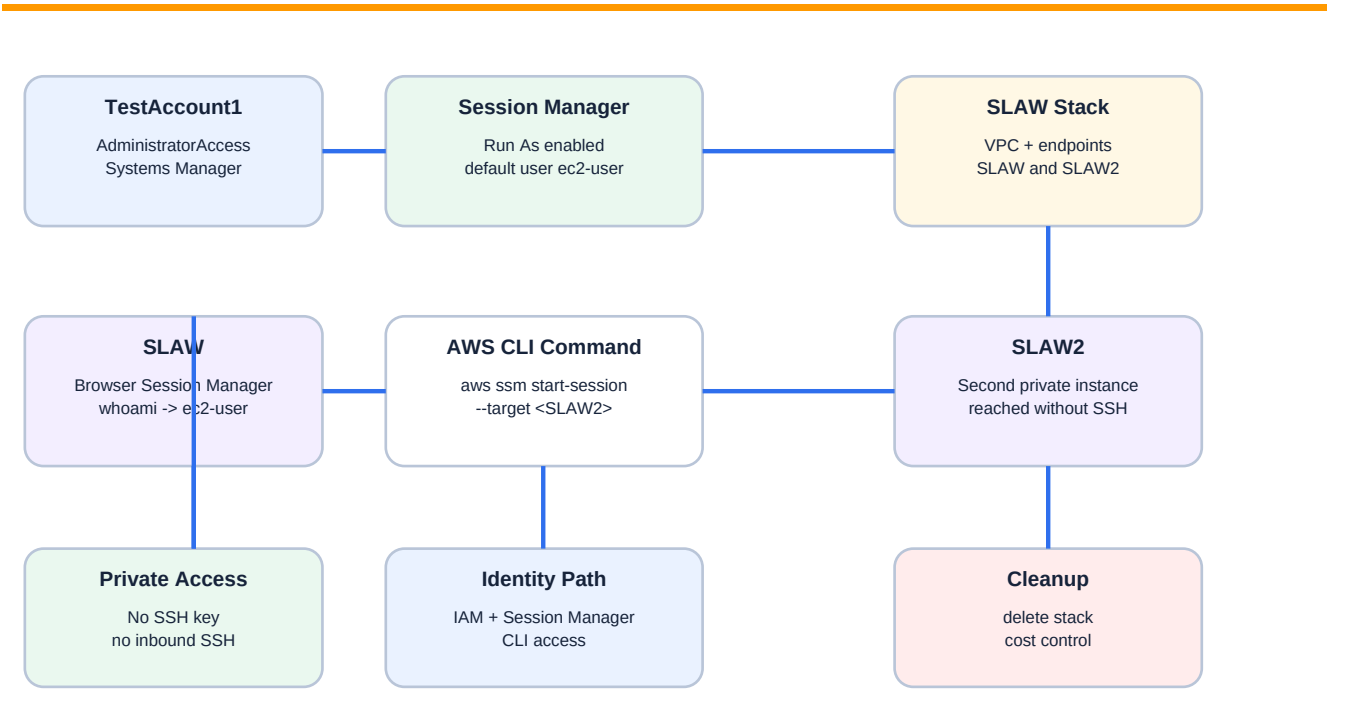
- Sign into TestAccount1 using AdministratorAccess.
- Open Systems Manager and navigate to Session Manager preferences.
- Enable Run As support for Linux instances.
- Set ec2-user as the default operating-system user for Amazon Linux Session Manager sessions.
- Create the SLAW CloudFormation stack from the provided lab template.
- Validate the stack-created SLAW and SLAW2 private EC2 instances.
- Connect to SLAW using Session Manager and confirm the shell runs as ec2-user.
- Use the AWS CLI from the SLAW session to run aws ssm start-session --target against the SLAW2 instance ID.
- Demonstrate that one private instance can start a Session Manager connection to another without SSH.
- Delete the CloudFormation stack after the lab for cost control.

Environment

Cloud Provider	Amazon Web Services (AWS)
Primary Services	AWS Systems Manager Session Manager, Amazon EC2, AWS CloudFormation

Account Context	TestAccount1 / AdministratorAccess
Stack Name	SLAW
Template Source	Provided CloudFormation template URL from the lab
Compute Pattern	Two private Amazon Linux EC2 instances: SLAW and SLAW2
Instance Type	t2.micro in evidence
Availability Zone	us-west-2a in evidence
Session Manager Preference	Run As support enabled for Linux instances
Default OS User	ec2-user
Validation 1	whoami returned ec2-user
Validation 2	aws ssm start-session --target started a new session
Cleanup	CloudFormation stack deleted after the lab for cost control

Architecture



Run As controls the OS user, while the AWS CLI can start a new Session Manager session to another private instance.

Figure 1 - Session Manager Run As and AWS CLI-based private instance access.

Implementation Summary

1. Signed into the AWS access portal and selected TestAccount1 with AdministratorAccess.
2. Opened AWS Systems Manager and selected Session Manager.
3. Opened Preferences and selected Edit.
4. Enabled Run As support for Linux instances.
5. Entered ec2-user as the operating system user name and saved the Session Manager preferences.
6. Created the SLAW CloudFormation stack from the provided template.
7. Waited for the stack and instances to finish provisioning.
8. Opened EC2 and confirmed SLAW and SLAW2 were running t2.micro instances in us-west-2a.
9. Connected to SLAW using the browser-based Session Manager connection path.
10. Ran whoami and confirmed the active operating-system user was ec2-user.
11. From the SLAW shell, ran aws ssm start-session --target followed by the SLAW2 instance ID.
12. Confirmed a new Session Manager session started from the command line.
13. Deleted the SLAW CloudFormation stack after the lab to remove cost-generating resources.

Command-Line Pattern

```
aws ssm start-session --target <SLAW2-instance-id>
```

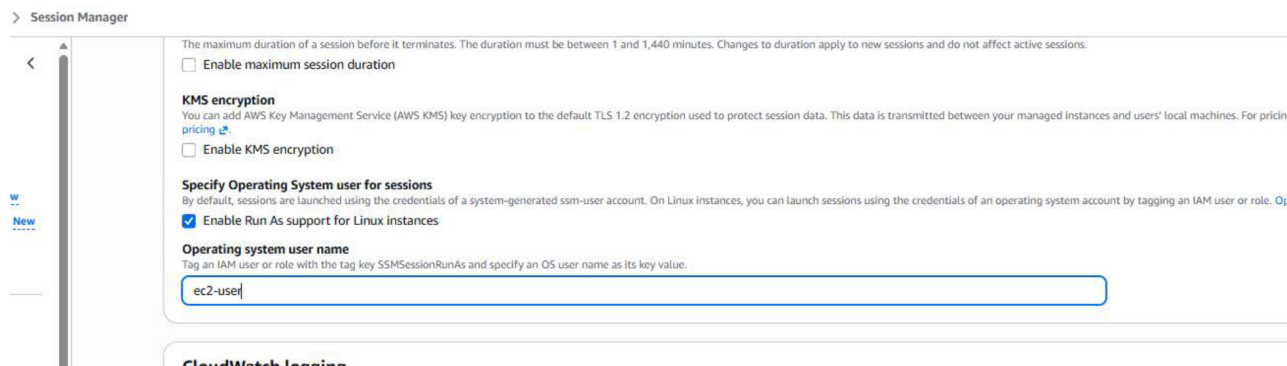
The command starts a Session Manager session to a managed EC2 instance by instance ID. In this lab, it was run from a Session Manager shell on SLAW to connect to SLAW2.

Control Map

Run As	Linux Run As enabled with ec2-user	Session Manager starts the shell as the intended Amazon Linux user instead of generic ssm-user.
Browser Session Manager	Connect to SLAW from console	Provides administrative shell access without inbound SSH.
AWS CLI Session Manager	aws ssm start-session --target	Demonstrates command-line access without SSH keys.
Private instances	SLAW and SLAW2 running without public IPv4 shown	Keeps workload administration inside managed AWS access paths.
CloudFormation stack	SLAW stack from provided template	Creates the VPC, endpoints, and instances consistently for lab testing.
Cleanup	Delete stack after lab	Prevents continued lab resource costs.

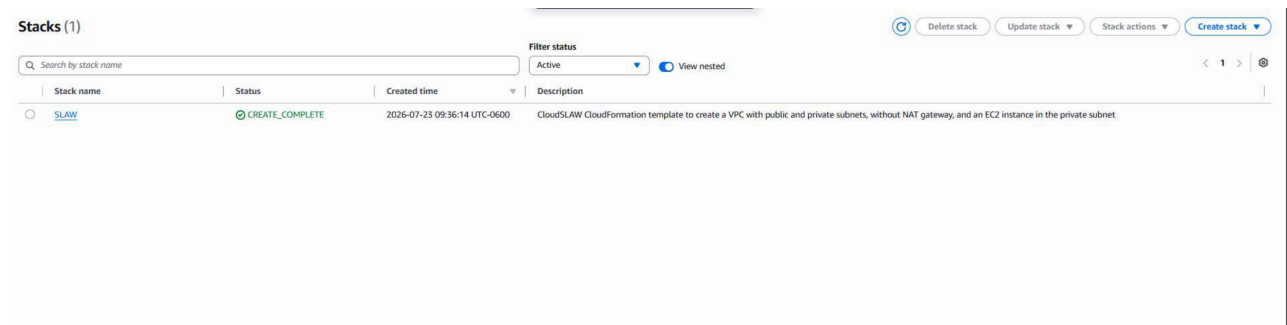
Evidence

Figure 2 - Session Manager Preferences Configured for ec2-user



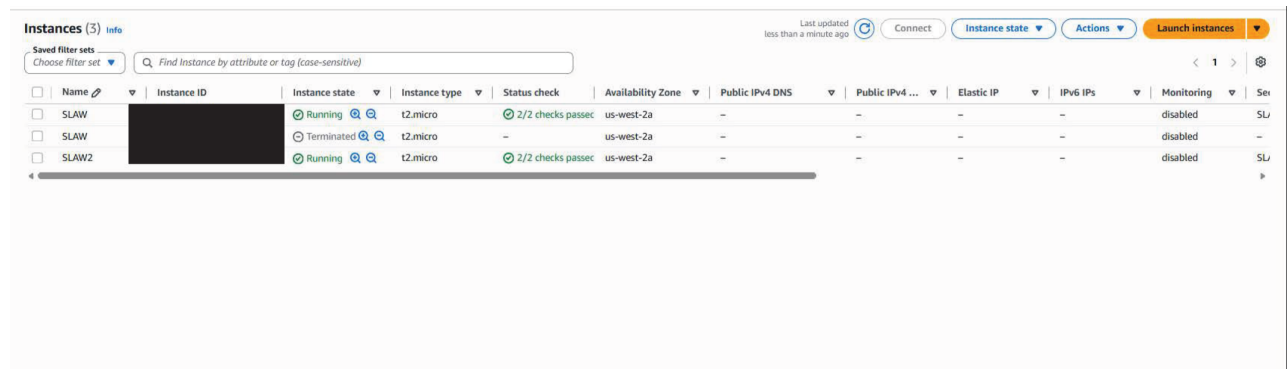
The Session Manager preferences evidence shows Run As support enabled for Linux instances and the operating system user name set to ec2-user. KMS encryption remains unchecked in this captured view.

Figure 3 - SLAW CloudFormation Stack Created



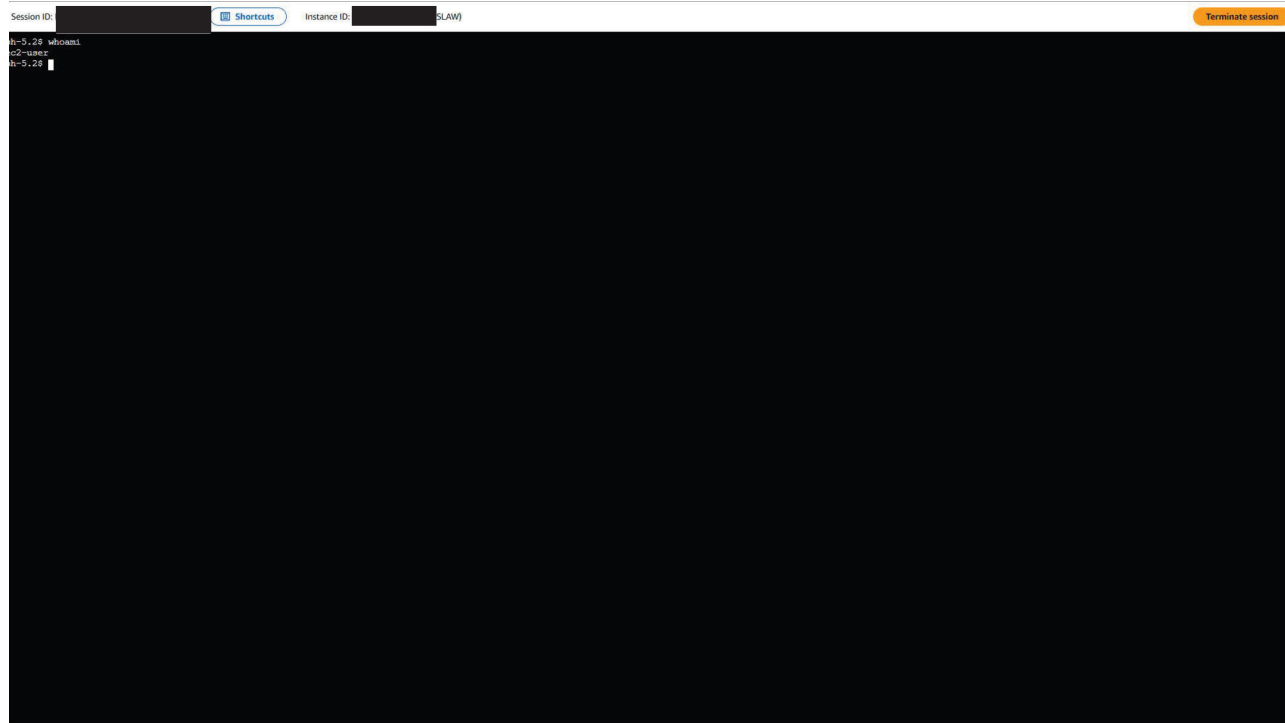
The CloudFormation evidence shows the SLAW stack with CREATE_COMPLETE status. The visible description indicates the stack creates a CloudSLAW VPC with public and private subnets, no NAT Gateway, and an EC2 instance in the private subnet.

Figure 4 - SLAW and SLAW2 Instances Running



The EC2 instance evidence shows SLAW and SLAW2 running as t2.micro instances in us-west-2a with 2/2 status checks passed and no public IPv4 values shown. A terminated earlier SLAW entry is also visible in the list. Instance IDs were redacted.

Figure 5 - Run As Validation Shows ec2-user



The Session Manager terminal evidence shows the whoami command returning ec2-user, confirming the Run As preference changed the Linux operating-system user for the Session Manager shell.

Figure 6 - AWS CLI Starts a Session Manager Session to SLAW2



The command-line evidence shows aws ssm start-session --target followed by a redacted SLAW2 instance ID. The terminal then shows Starting session with SessionId, confirming the CLI-based Session Manager connection method worked.

Security Analysis

This project further reduces dependency on SSH by showing both console-based and command-line Session Manager access paths.

Run As improves operating-system-level control because sessions can start as a designated Linux user rather than the default ssm-user. This helps separate AWS access from OS privilege decisions.

The CLI start-session workflow allows operators and developers to use terminal workflows without storing SSH private keys or exposing port 22.

Session Manager access can be controlled with IAM policies and can integrate with MFA through IAM Identity Center role access, unlike unmanaged SSH keys.

The design still requires careful IAM scoping. AdministratorAccess works for the lab, but production should limit who can start sessions and which instances can be targeted.

The SLAW-to-SLAW2 test relies on IAM permissions and Systems Manager connectivity. It is not direct SSH between instances.

Lessons Learned

Session Manager can replace common SSH workflows for Linux instance administration.

Run As controls the Linux account used inside the operating system.

The aws ssm start-session command allows command-line Session Manager access.

A browser Session Manager session can be used as a consistent shell environment when local AWS CLI tooling is not available.

Two private instances can be managed through the Systems Manager control plane without assigning public IPs or opening inbound SSH.

Production Improvements

- Restrict ssm:StartSession permissions by instance tag, account, and environment.
- Enable Session Manager logging to S3 or CloudWatch Logs for command accountability.
- Enable KMS encryption for session data and logs.
- Use IAM Identity Center permission sets instead of broad AdministratorAccess for routine operations.
- Define Run As behavior through tags for different roles and instance groups.
- Create VPC endpoints for Systems Manager services in multiple Availability Zones.
- Monitor StartSession events with CloudTrail and EventBridge.
- Use documented break-glass access for emergency situations rather than leaving SSH open.

Skills Demonstrated

Session Manager	Run As support	AWS CLI start-session
Amazon EC2	CloudFormation stack lifecycle	Private instance management
No SSH key access	OS user validation	Cost-control cleanup

Resume Bullet

RESUME	Configured AWS Systems Manager Session Manager Run As support for Amazon Linux, validated ec2-user shell access, and demonstrated command-line Session Manager connectivity from one private EC2 instance to another using aws ssm start-session without SSH keys or inbound SSH exposure.
--------	--

STAR Interview Story

Situation: The lab already used Session Manager for private instance access, but the default workflow still needed better operating-system user control and command-line usability.

Task: Configure Session Manager to use the Amazon Linux ec2-user account and prove that Session Manager can be launched from the command line as an SSH alternative.

Action: I enabled Run As support for Linux instances, set ec2-user as the default user, created the SLAW stack, connected to SLAW, verified whoami returned ec2-user, and used aws ssm start-session --target from SLAW to connect to SLAW2.

Result: The lab demonstrated browser and CLI Session Manager access to private EC2 instances without SSH keys, public IPs, or inbound SSH rules.

Version History

v1.0.0	July 23, 2026	Initial documented implementation. Enabled Session Manager Run As support for Linux instances, set ec2-user as the default OS user, created SLAW stack, validated SLAW/SLAW2 instances, confirmed ec2-user shell access, demonstrated CLI-based aws ssm start-session to SLAW2, documented cleanup, and embedded provided evidence using portfolio documentation standard v1.0.1.

References

- Cloud Security Lab a Week (S.L.A.W.) - Replace SSH with Session Manager.
- User-provided Session Manager preferences evidence screenshot.
- User-provided CloudFormation SLAW stack evidence screenshot.
- User-provided EC2 instances evidence screenshot.

- User-provided ec2-user Run As validation screenshot.
- User-provided AWS CLI Session Manager connection screenshot.
- Project 32 - Launch a Private Amazon Linux EC2 Instance with Session Manager.
- Project 33 - Keep Private Subnets Private with VPC Endpoints.